

Integración de Servicios Cloud GCP

Servicios, sockets y modelo cliente-servidor

Semana 04 - Sesión 07

Rel Guzman



**Universidad
Tecnológica
del Perú**

- ▶ ¿Ya instalaron Visual Studio Code, Warp y Antigravity CLI?
- ▶ ¿Qué comando usaron para encontrar la puerta de enlace de la VM?
- ▶ ¿Qué protocolo y puerto encuentran al abrir `http.rip` en el navegador?

Logro de la sesión

Al finalizar, podrás explicar cómo un servicio atiende clientes, describir qué es un socket y los pasos que siguen servidor y cliente, escribir en Python un servidor y un cliente HTTP mínimos con sockets, y averiguar en el sistema qué proceso ocupa un puerto.

- ▶ ¿Se acuerdan de las capas del modelo OSI?
- ▶ ¿Ya crearon servidores en Node.js o Python?

¿Para qué te servirá lo aprendido?

Casi todo servicio cloud que uses —una API, una base de datos, un balanceador de carga— es un programa escuchando en un socket. Escribir uno pequeño te permite entender qué ocurre cuando algo no responde.

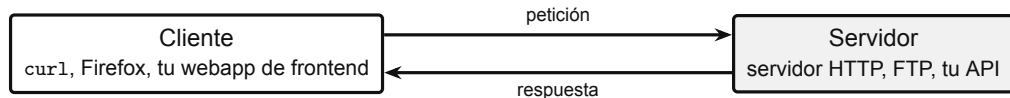
También podrás leer una petición y una respuesta HTTP línea por línea, y reconocer los errores más comunes al levantar un servicio: `Address already in use` y `Connection refused`.

Servicios, sockets y modelo cliente-servidor

Un servicio es un programa que espera peticiones

Idea central

El **cliente** inicia la conversación y pide algo. El **servidor** no hace nada hasta que alguien le pide: permanece a la espera, atiende y vuelve a esperar.



Servicio o *daemon*

Un programa que el sistema mantiene corriendo en segundo plano, sin ventana ni terminal propia, administrado por `systemd`.

Un mismo equipo hace ambos papeles

Tu VM es servidor cuando publica una API y cliente cuando consulta `google.com`.

El socket es el punto donde se atiende la conexión

Qué lo define

Un socket queda identificado por tres datos: **protocolo** (TCP o UDP), **dirección IP** y **puerto**. Por eso una misma IP puede atender muchos servicios a la vez.

Los cuatro pasos del servidor

```
socket() crea el punto de conexión  
bind() lo fija a IP y puerto  
listen() abre la cola de espera  
accept() atiende a un cliente
```

La dirección de escucha importa

127.0.0.1 solo acepta clientes de la propia máquina.

Un puerto, un servicio

Si dos programas piden el mismo puerto, el segundo falla con Address already in use.

Por qué se puede escribir a mano

HTTP no tiene nada de mágico: el cliente envía líneas de texto y el servidor devuelve otras líneas de texto. Lo único obligatorio es respetar el formato: una línea en blanco separa las cabeceras del contenido.

Lo que envía el cliente

```
GET / HTTP/1.1  
Host: 127.0.0.1:8080  
(línea en blanco)
```

Método, ruta y versión; luego las cabeceras.

Lo que responde el servidor

```
HTTP/1.1 200 OK  
Content-Type: text/plain  
Content-Length: 41  
(línea en blanco)  
Hola desde un servidor...
```

Código de estado, cabeceras y contenido.

Núcleo del script

```
import socket
s = socket.socket(socket.AF_INET,
    socket.SOCK_STREAM)
s.bind(('127.0.0.1', 8080))
s.listen(5)
while True:
    cli, dir = s.accept()
    cli.recv(1024)
    cli.sendall(Respuesta.encode())
    cli.close()
```

Pruébalo

```
$ python3 servidor_http.py
Escuchando en
http://127.0.0.1:8080

$ curl -i 127.0.0.1:8080
HTTP/1.1 200 OK
Hola desde un servidor
hecho con sockets
```

1. Con el servidor corriendo, pregunta a agy en otra terminal

Tengo servidor_http.py escuchando en el puerto 8080. Dame los comandos de Ubuntu para ver qué puertos están en escucha y qué proceso abrió cada uno, averiguar el PID que ocupa el 8080, y mostrar el nombre y las direcciones IP de esta máquina. Explica qué significa cada columna de la salida.

2. Anota lo que encuentres

```
PUERTO    = 8080
PID       = [? ]
PROCESO   = [? ]
DIRECCION = [? ]
HOSTNAME  = [? ]
```

Junto a cada dato, anota el comando que usaste y pide a agy que lo explique.

3. Detén el proceso por su PID

```
$ kill [PID ]
$ curl 127.0.0.1:8080
curl: (7) Failed to connect
```

Si no termina, kill -9 lo fuerza. Busca de nuevo el puerto: ya no debe aparecer.

Qué debes poder explicar

Por qué el 8080 solo acepta conexiones de la propia máquina, y qué cambia al buscar el puerto antes y después del kill.

Núcleo del script

```
import socket
c = socket.socket(socket.AF_INET,
    socket.SOCK_STREAM)
c.connect(('127.0.0.1', 8080))
c.sendall(PETICION.encode())
respuesta = c.recv(1024)
c.close()
print(respuesta.decode())
```

Es lo mismo que hace curl, escrito a mano.

El cliente hace menos pasos

```
socket() crea el punto
connect() busca al servidor
sendall() envía la petición
recv() lee la respuesta
```

No hay bind() ni listen(): el cliente no espera a nadie.



gracias



Universidad
Tecnológica
del Perú

- ▶ Documentación de Python: módulo `socket` y guía `Socket Programming HOWTO`, docs.python.org/3/library/socket.html.
- ▶ MDN Web Docs: *An overview of HTTP*, developer.mozilla.org/docs/Web/HTTP.
- ▶ RFC 9110 y RFC 9112: semántica de HTTP y HTTP/1.1 sobre TCP.
- ▶ Páginas de manual de `ss`, `kill`, `hostname` y `curl`.
- ▶ Documentación de Ubuntu Server, documentation.ubuntu.com/server.
- ▶ Documentación oficial de Google Antigravity CLI, antigravity.google/docs/cli.
- ▶ Material base proporcionado: *Fundamentos de Linux — Programa completo*; identidad visual y estructura docente de la Sesión 05.